Week 7: One- and two-dimensional DP

Repeated subproblem calculations

What information is needed to maximize inspection value when adjacent items cannot both be selected? Grid paths and common order in two event logs can also be divided into smaller problems.

Store and reuse previously computed optima or counts. The meaning of the state determines whether to add, minimize, or maximize.

Week 7 learning outcomes

Sessions 13 and 14: State meaning, base cases, recurrences, and evaluation order

- Check contracts and boundary conditions.
- Trace state and implement in Python.
- Explain correctness, time, auxiliary space, and output space.

Opening scenarios, traces, and checks are additional teaching examples.

Session 13 · Optimal subproblems and one-dimensional DP

Replace repeated prefix or amount computations with states. Define the optimization objective before assigning meanings to table entries.

Prerequisites and Python semantics

`float("inf")` represents unreachable minimization states. Tuple assignment evaluates the right side using old values; preserve both predecessor states.

Before allocating a DP table

- State: which answer for which part of the input?
- Base cases: empty input, amount zero, first row/column
- Recurrence: split cases by the final choice
- Evaluation order: compute dependencies first

Filling coin amounts downward reads smaller states before computing them. Positive coins permit increasing-amount evaluation.

198 · Problem and state

LeetCode 198: House Robber · Medium

Maximize selected value without selecting adjacent positions.

`nums=[3,1,4,2] → 7`

D[i] is the best prefix of length i. D[0]=0, D[1]=nums[0]; for i>=2, D[i]=max(D[i-1],D[i-2]+nums[i-1]). The code starts both rolling states at zero and applies the update from the first value.

198 · Implementation

```
def rob(nums):
    previous_two = previous_one = 0
    for value in nums:
        previous_two, previous_one = previous_one, max(previous_one, previous_two + value)
    return previous_one
```

198 · Correctness and cost

Every optimum either selects the current item or skips it. Each case leaves an optimal prefix, so the recurrence covers all optima.

n is length. O(n) time, O(1) auxiliary space. Reconstructing selected indices needs additional state.

322 · Problem and state

LeetCode 322: Coin Change · Medium

Return the minimum coin count for amount, or -1 if impossible.

`coins=[1,3,4], amount=6 → 2`

Set dp[0]=0 and other states to infinity; fill increasing amounts using min(dp[a-c]+1).

322 · Implementation

```
def coin_change(coins, amount):
    dp = [float("inf")] * (amount + 1)
    dp[0] = 0
    for total in range(1, amount + 1):
        for coin in coins:
            if coin <= total:
                dp[total] = min(dp[total], dp[total - coin] + 1)
    return -1 if dp[amount] == float("inf") else dp[amount]
```

322 · Correctness and cost

Removing the last coin c from an optimum leaves an optimum for a-c. Comparing all final coins finds the minimum; positive values ensure dependencies were filled earlier.

A is amount,m coin types. O(Am) time,O(A) space; pseudopolynomial in the bit length of A.

State trace

a	0	1	2	3	4	5	6
dp[a],coins=1,3,4	0	1	2	1	1	2	2

Each amount uses already-computed smaller amounts.

Failure inputs and concept checks

Summing only even or only odd positions misses optimum 4 for [2,1,1,2].

Why add one to dp[a-c]?

The instructor answer is in the separate guide. Compare the reasoning line by line with the companion implementation.

Session 14 · Grid and prefix-string DP

Count grid paths and compare event sequences. A table shape does not determine whether transitions should sum or maximize.

Prerequisites and Python semantics

Allocate independent rows with a comprehension. A prefix of length i ends at first[i-1]; reserve row/column zero for empty prefixes.

62 · Problem and state

LeetCode 62: Unique Paths · Medium

Count grid paths using only right and down moves.

`rows=2, cols=4 → 4`

Initialize first row/column to 1, add top and left counts, and allocate each row independently.

62 · Implementation

```
def unique_paths(rows, cols):
    dp = [[1] * cols for _ in range(rows)]
    for r in range(1, rows):
        for c in range(1, cols):
            dp[r][c] = dp[r-1][c] + dp[r][c-1]
    return dp[-1][-1]
```

62 · Correctness and cost

Every path arrives from top or left. These disjoint classes cover all paths, so counts add; row order respects dependencies.

R,C are dimensions. O(RC) time/space under the bounded-answer model; arbitrary-size integer bit costs are separate.

1143 · Problem and state

LeetCode 1143: Longest Common Subsequence · Medium

Return the length of a longest common subsequence preserving order.

`first="abca", second="aca" → 3`

Equal final characters give diagonal+1; otherwise take max(top,left); empty prefixes give zero.

1143 · Implementation

```
def lcs(first, second):
    rows, cols = len(first), len(second)
    dp = [[0] * (cols + 1) for _ in range(rows + 1)]
    for r in range(1, rows + 1):
        for c in range(1, cols + 1):
            if first[r-1] == second[c-1]:
                dp[r][c] = dp[r-1][c-1] + 1
            else:
                dp[r][c] = max(dp[r-1][c], dp[r][c-1])
    return dp[rows][cols]
```

1143 · Correctness and cost

Equal final characters can be paired in an optimum. Unequal ends cannot both be the final match, so at least one is skipped; maximize over both cases.

n,m are string lengths. O(nm) time/space; this returns length, not a reconstructed sequence.

State trace

LCS(“ab”,”ac”)	empty prefix	a	c
empty prefix	0	0	0
a	0	1	1
ab	0	1	1

Rows and columns denote prefix lengths, including zero.

Failure inputs and concept checks

`[[1]*C]*R` aliases rows; use a comprehension creating each row.

Why add rather than maximize when counting paths?

The instructor answer is in the separate guide. Compare the reasoning line by line with the companion implementation.

Preserving two previous optima

Current value	Old prev_two	Old prev_one	New optimum
3	0	0	3
1	0	3	3
4	3	3	7
2	3	7	7

Every right-hand expression uses the old state. Selecting the current value skips the immediately preceding position.

Checkpoint questions

1. Find the maximum nonadjacent sum for `[2,1,1,2]`.
2. Find the minimum coins for amount 6 using `[1,3,4]`.
3. Count paths in 1×4 and 2×4 grids.
4. Find the LCS length of `abc` and `ac`. Must the characters be contiguous?

Independent practice and connections

Student exercises

Hints and solutions

Instructor guide

Download Week 7 code

Submit code, state definition, correctness argument, complexity, and boundary inputs. Week 8 includes a mixed assessment without pattern labels.

Execution and sources

Run in the `code` folder of the weekly code archive.

```
python demo_13.py
python demo_14.py
python verify.py
python verify.py --student
```

Use the last command after implementing the student file.

Erik Demaine and Srini Devadas, MIT 6.006: Introduction to Algorithms (2011).